



EGH446 GROUP PROBLEM SOLVING TASK (PST)

Part A: Guidance (Individual)



14 SEPTEMBER 2024
RORY BLAIR, N10773398
GROUP 19

1). Overview

1.1). Design

This report will show the initial stages of creating a ground vehicle that utilises an autonomous system to control its movements. This autonomous system is created by designing three different sub-systems that operate together but can operate independently.

The 1st and most important sub-system is the 'Guidance and Control' sub-system, which is responsible for planning and controlling the path the vehicle takes, making this a closed-loop system. This system employs the use of a Receding Virtual Waypoint (RVWP) guidance system, which shows a point directly in front of the vehicle which it then follows to proceed along the correct path [1].

The 2nd sub-system is the waypoint optimisation system, which runs multiple path optimisation algorithms to discover which algorithms would optimise either time or distance. The different algorithms tested are the unoptimized, Brute Force (BF), Branch & Bound (BnB), and two versions of the Nearest Neighbour (NN).

The final subsystem is the filtering system, which is a closed-loop system that will filter out noise that has been generated as 0.1m for 'x' and 'y' coordinates, and 2 degrees for angles. This noise is filtered out of the desired signal by using a Kalman filter. A Kalman filter is a recursive filter as it is used to find the system state and its optimal estimate. This filter is also used to minimise the mean square error by incorporating all of the data available into an estimate [2].

This report will investigate the first two subsystems mentioned, the 'guidance/control' system, and the 'waypoint optimisation' system, and the many algorithms tested and utilised. This will include creating the sub-systems, testing them separately, and then testing them together to show that the overall system can work. The overall system is shown in Figure 1.1 below, which shows the overall system the system's inputs and output.

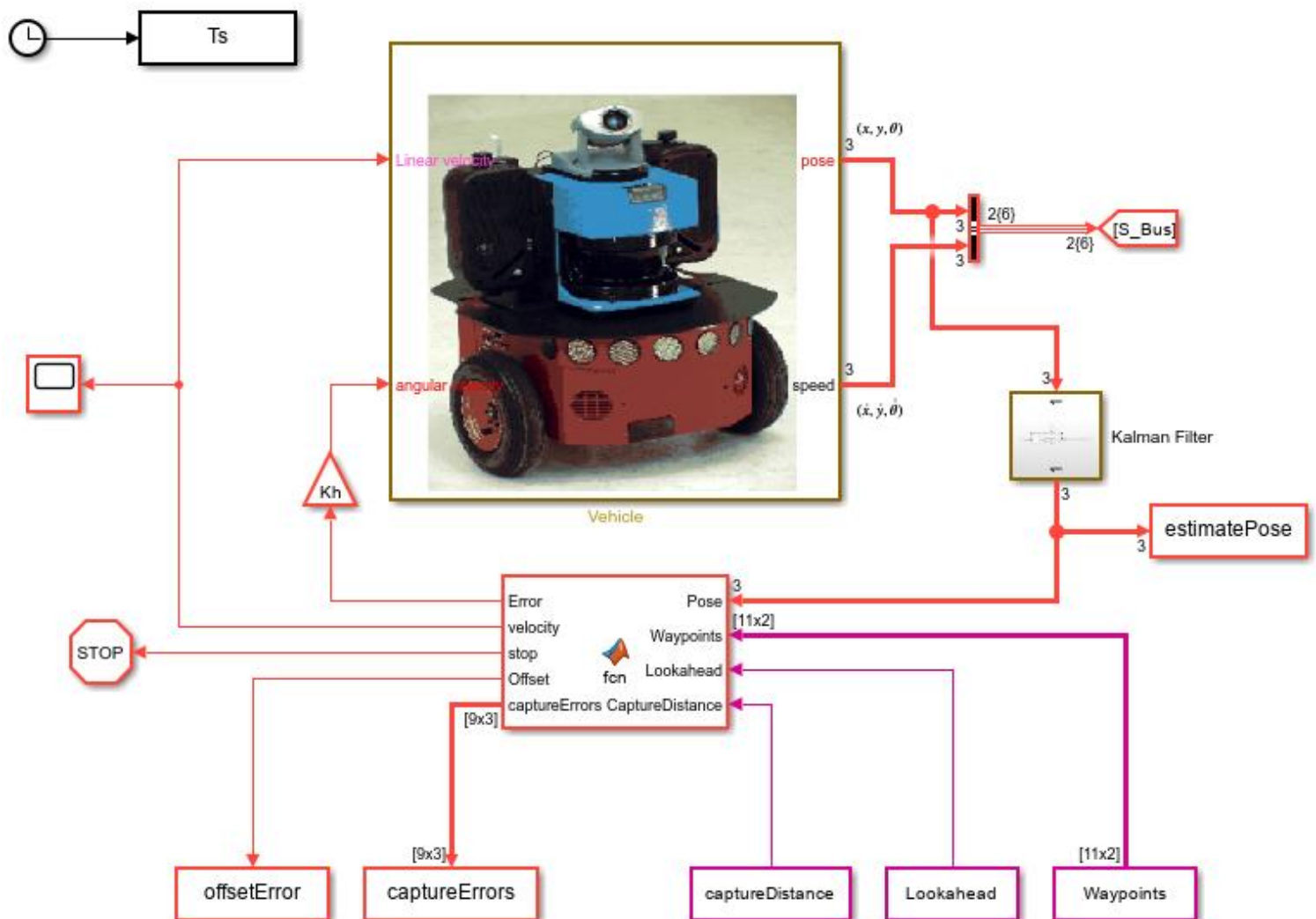


Figure 1.1: Overall System Diagram (Simulink MATLAB)

1.2). Inputs and Outputs

As seen in the diagram above, the vehicle takes two inputs called 'linear velocity' and 'angular velocity', which is the speed that is given by the user. The vehicle will then give three outputs the 'x' and 'y' coordinates as well as theta, which is the angle the vehicle currently faces. These three outputs are then used as input into the RVWP system, which will use the values and waypoints given to calculate the new angular velocity and the linear velocity. the angular velocity is used to change the angle the vehicle is taking, thus making sure the vehicle is following the desired route. The linear velocity is used to change the speed of the vehicle, which allows for no overshoot of the waypoints. This process is repeated until all waypoints have been met.

2). Guidance and Control

2.1). Vehicle Controls

The controls are directed by two inputs that are given to the vehicle called the linear velocity and the angular velocity. In this assignment, the linear velocity and angular velocity change based on the output provided by the RVWP system. The vehicle will output the 'x', 'y', and 'theta' coordinates which are then used as the input for the RVWP system. This entire process is repeated continuously until the vehicle reaches the final waypoint provided. The vehicle's inner workings can be seen in a diagram below in Figure 2.1.

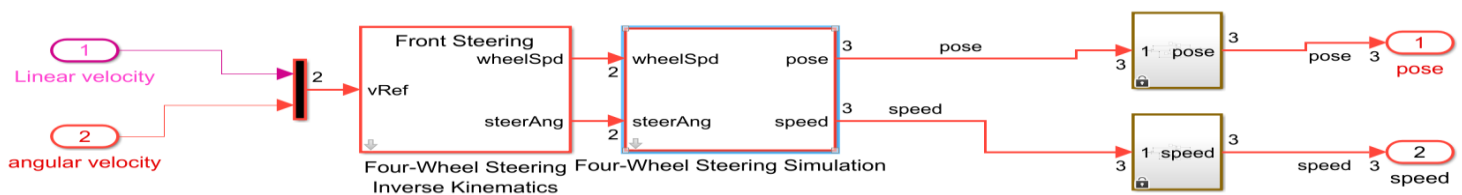


Figure 2.1: Vehicle control system (Simulink MATLAB)

As seen above, the system takes the two inputs and puts them into an inverse kinematics system. An inverse kinematics system finds the gains of control variables given the desired end arrangement [3]. The output from the kinematics system is called 'Wheel Speed' and 'Steer angle', which is then used as input for the steering simulation system, which in turn is used to output the 'x', 'y', and 'theta' variables for the vehicle to follow. The RVWP system then uses these variables to make changes to the vehicle's angular and linear velocity's if needed. This RVWP system is the box below the picture of the vehicle (see figure 1.1), which contains the function called 'fcn'. This function is used as the RVWP system to calculate and alter the vehicle's course. The code for this 'fcn' function can be seen in Figure 2.2 below.

```

function [Error, stop, Offset, captureErrors, velocity] = fcn(Pose,
Waypoints, Lookahead, CaptureDistance, time)
    persistent waypointIdx
    persistent capErrors
    if isempty(waypointIdx)
        waypointIdx = 2;
        capErrors = [ones(size(Waypoints,1)-1,1)*CaptureDistance,
zeros(size(Waypoints,1)-1, 2)];
    end
    Mode = "RVWP";
    if Mode == "Direct"
        Target = transform(Pose, Waypoints(WaypointIdx,:));
        %Computing distance from target point
        Distance = sqrt((Waypoints(WaypointIdx,1) -
X)^2+(Waypoints(WaypointIdx,2) - Y)^2);
        Last = transform(Pose, Waypoints(WaypointIdx-1,:));
        RW1 = sqrt((Last(1))^2+(Last(2))^2);
        RW2 = sqrt((Target(1))^2+(Target(2))^2);
        W1W2 = sqrt((Target(1)-Last(1))^2+(Target(2)-Last(2))^2);
        cosAlpha = (W1W2^2 + RW1^2 - RW2^2) / (2 * W1W2 * RW1);
        cosAlpha = max(min(cosAlpha, 1), -1);
        Alpha = acos(cosAlpha);
        Offset = sin(Alpha)*RW1;
    elseif Mode == "RVWP"
        [Target, Distance, Offset, capOffset] = RVWP(Pose, Waypoints,
waypointIdx, Lookahead);
    end
    if capOffset < capErrors(waypointIdx-1,1)
        capErrors(waypointIdx-1,1) = capOffset;
        capErrors(waypointIdx-1,2:3) = Pose(1:2);
    end
    captureErrors = capErrors(2:end, :);
    Error = atan2(Target(2), Target(1));
    velocity = max(min([2.5+2.5*(Distance-100)/capOffset,
2.5+2.5*(capOffset-100)/Distance, 5]),2.5);
    stop = 0;
    if Distance < CaptureDistance
        if waypointIdx < size(Waypoints,1)
            waypointIdx = waypointIdx + 1;
        else
            stop = 1;
        end
    end
end
end

```

Figure 2.2: 'fcn' function (MATLAB)

This function starts by creating persistent variables so that the function can keep track of the current waypoint index. The function then initialises the navigation variables and then creates the distance and target parameters. The function then uses the RVWP function to compute the target distance. The function then computes the navigation error and initialises the stop point. The function finally checks to see if the current waypoint has been met yet.

The input for this function is 'Pose', 'Waypoints', 'Lookahead', and 'CaptureDistance'. The 'Pose' variable is the current configuration of the vehicle, such as 'x', 'y' and 'theta'. 'Waypoints' is a matrix where the rows represent a single waypoint. The 'Lookahead' is the distance which the vehicle should scan ahead of the path to determine the next action. The 'CaptureDistance' is the distance which the vehicles considers a waypoint as captured.

The outputs for this function are the 'Error', 'Stop', 'Offset', 'captureErrors' and 'velocity'. The 'Error' variable is the representation of the angle navigation error and the 'stop' variable is a flag that will tell the navigation to stop or to continue. The 'Offset' is the distance the vehicle is from being on the desired path. The 'captureErrors' is an array that keeps track of the number of capture point errors. The 'velocity' is the newly calculated velocity the vehicle should be at.

2.2). Receding Virtual Waypoint (RVWP)

The Receding Virtual WayPoint (RVWP) system is the system that is used to always check the position and angle of the vehicle. The code for the RVWP system is seen in Figure 2.3 below. (See Appendix I for 'transform' function code)

```
function [Target, RW2, Offset, RW1] = RVWP(RobotPose, Waypoints,
WaypointIdx, Lookahead)
    lastWaypoint = transform(RobotPose, Waypoints(WaypointIdx-1,:));
    targetWaypoint = transform(RobotPose, Waypoints(WaypointIdx,:));
    RW1 = sqrt((lastWaypoint(1))^2+(lastWaypoint(2))^2);
    RW2 = sqrt((targetWaypoint(1))^2+(targetWaypoint(2))^2);
    W1W2 = sqrt((targetWaypoint(1)-lastWaypoint(1))^2+(targetWaypoint(2) -
lastWaypoint(2))^2);
    cosAlpha = (W1W2^2 + RW1^2 - RW2^2) / (2 * W1W2 * RW1);
    cosAlpha = max(min(cosAlpha, 1), -1);
    Alpha = acos(cosAlpha);
    Distance = cos(Alpha)*RW1;
    Offset = sin(Alpha)*RW1;
    Beta = atan2(targetWaypoint(1) - lastWaypoint(1), targetWaypoint(2) -
lastWaypoint(2));
    Target = [lastWaypoint(1) + (Distance + Lookahead)*sin(Beta),
lastWaypoint(2) + (Distance + Lookahead)*cos(Beta)];
end
```

Figure 2.3: Code for RVWP function (MATLAB)

This code starts by transforming the waypoints from the format given to a format the vehicle's coordinate system can comprehend. This function then computes the distance from other waypoints to the robot and then the distance between waypoints. After this has been calculated, the program then finds the angles between waypoints. The next part then calculates the target waypoint taking the lookahead distance into consideration. The next part calculates the vehicle's position relative to the origin position of the vehicle. The final section converts the targets point from their current format to the global system for coordinates.

The inputs of this function are the variables called 'RobotPose', 'Waypoints', 'WaypointIdx', and 'Lookahead'. The 'RobotPose' variable is the vehicle's current pose, and the 'Waypoint' variable is the waypoints in a matrix format. The 'WaypointIdx' is the current waypoint index and the 'Lookahead' variable is the distance ahead to look for a waypoint.

The outputs for this function are called 'Target', 'RW2', 'Offset' and 'RW1'. The 'Target' variable is the target waypoint for the vehicle, based on the current waypoints, previous waypoints, lookahead distance, and vehicle pose. The 'RW2' variable is the length from the target waypoint to the vehicle's current location. The 'Offset' variable perpendicular distance from the vehicle's trajectory to the desired trajectory/path. The 'RW1' variable is the length from the target waypoint to the vehicle's previous location.

2.3). Testing Linear Velocity

Although the linear velocity is constant throughout the simulation, changing the value of the linear velocity will affect how the vehicle connects these waypoints. A representation of this effect can be seen in Figure 2.4 below. This figure shows the linear velocity at 1(left) and 5(right), and the angular velocity is 0.1. The linear velocity affects not only how fast the vehicle completes the path but also the amount the vehicle will overshoot the waypoint and the distance it takes to turn.

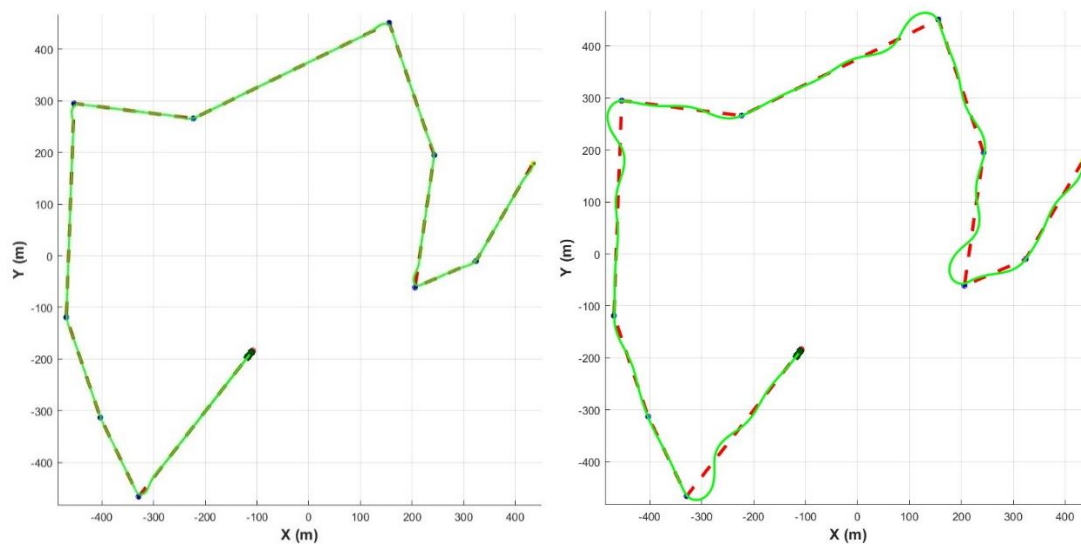


Figure 2.4: Testing effects of linear velocity

2.4). Testing Angular Velocity

Although the angular velocity is always being changed due to the RVWP system, the 'K' variable that is combined with the output can also be manually changed. The effects of changing the angular velocity can be seen in Figure 2.5 below. The left side is when the angular velocity is 0.1 and the right is when the angular velocity is 10. As seen in the figure below, when the angular velocity is 0.1 the vehicle will pass through the waypoint. However, when the angular velocity is 10, the vehicle will tend to turn before passing over the waypoint. This scenario is considered as a waypoint capture error and can be clearly seen on the far right waypoint. [4]

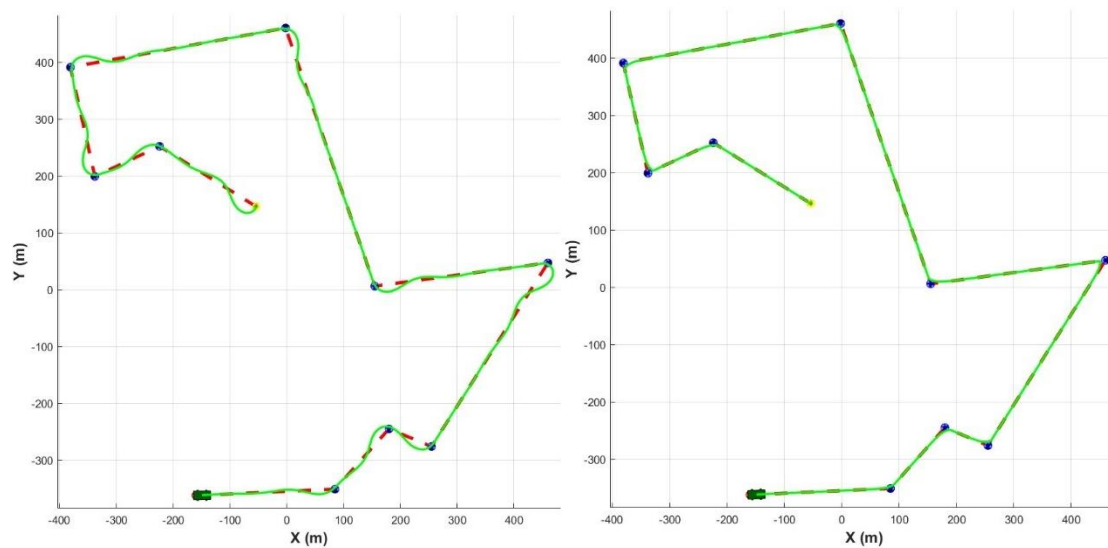


Figure 2.5: Testing effects of angular velocity

2.5). Testing Mission Time

The time to complete the mission is determined by many factors, such as distance/route, waypoint placement, waypoint capture errors and offset errors. The main factors that change the mission completion time are the distance and waypoint placement. Although the waypoint placement cannot be controlled, the distance and/or the route the vehicle takes can be controlled. This would allow for the mission time to be affected by the overall distance the vehicle completes. An example of this can be seen in Figure 2.6 below, which displays a small route (Top) and a massive route (Bottom). As seen below, the total distance travelled does affect the mission time, as the vehicle would take another 3 minutes to travel an extra 700m.

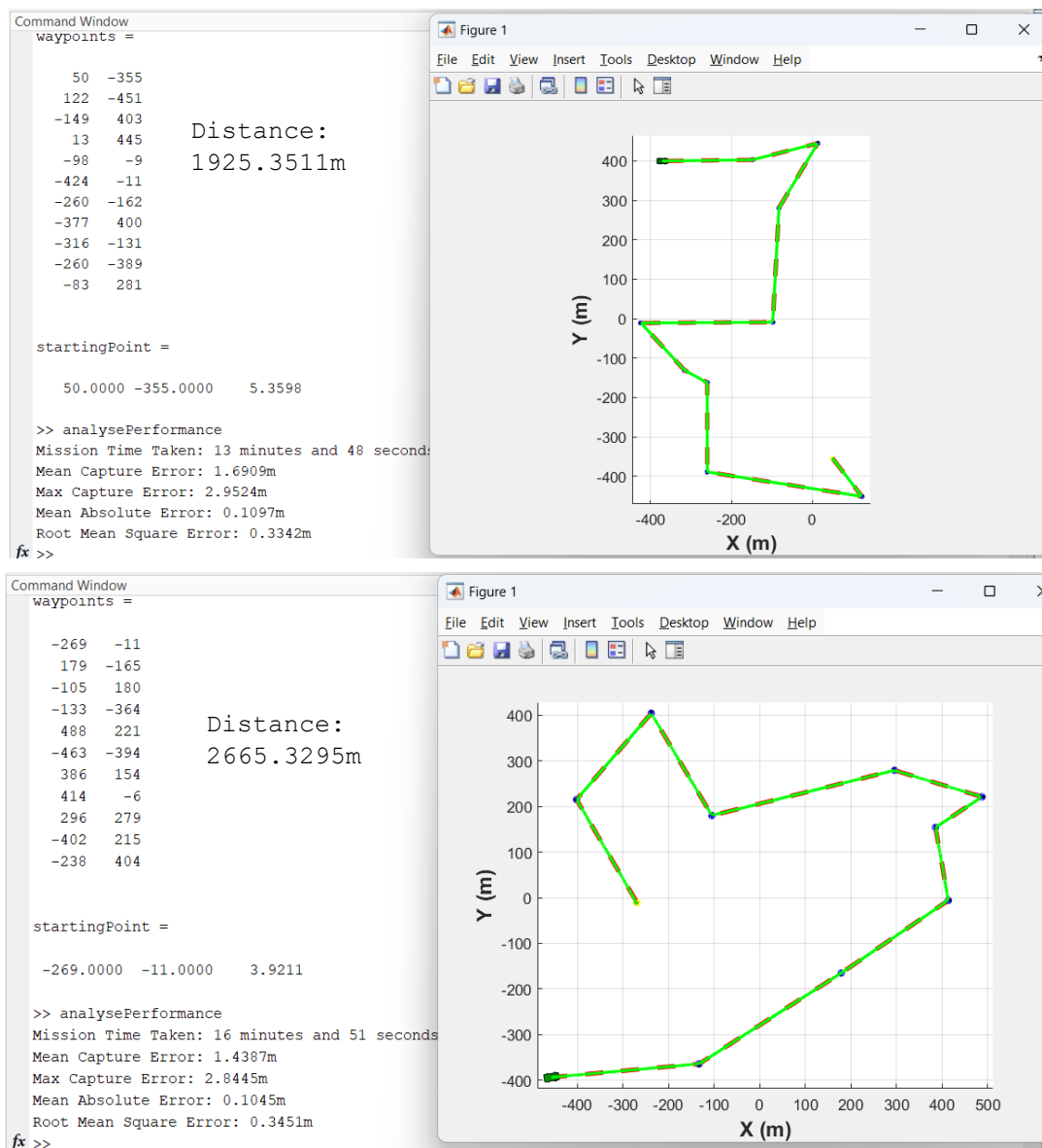


Figure 2.6: Testing effects of distance for mission time

2.6). Testing Waypoint Capture Error

The waypoint capture error is when a robot vehicle considers a waypoint capture even though it has not fully crossed over the waypoint. A waypoint is captured based on a set distance from the vehicle to the waypoint. The waypoint capture error would improve on mission completion time as the vehicle would not be overshooting, thus less distance. This trade-off between mission completion time and waypoint capture error is sufficient/acceptable. An example of the waypoint capture error can be seen in Figure 2.7 below.

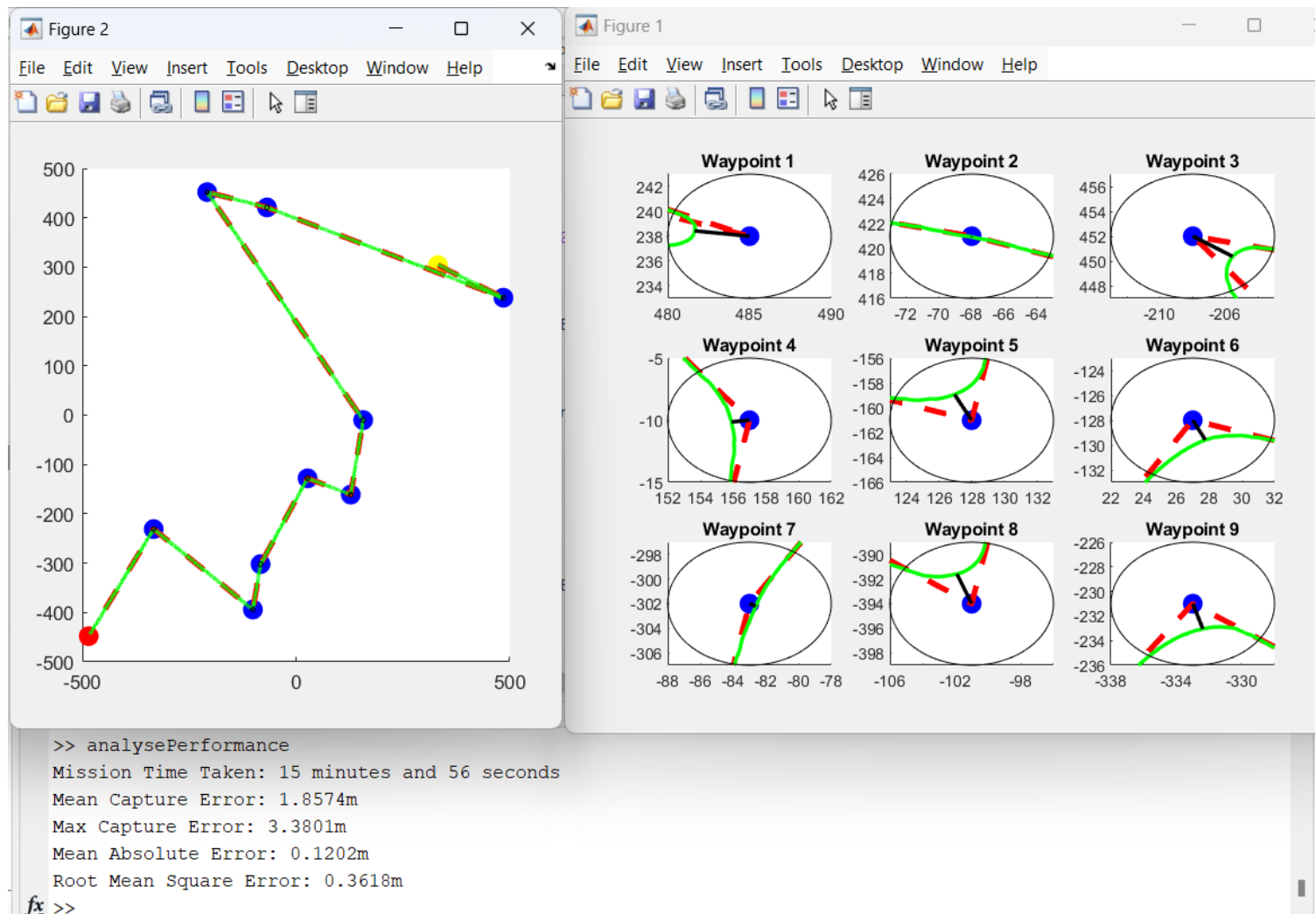


Figure 2.7: Testing effects of angles for waypoint capture error

As shown below, the waypoint capture error is clear and that waypoints that require the vehicle to turn more than 90 degrees, tend to turn earlier. The greater the vehicle has to turn, the earlier the vehicle starts to turn, thus a greater waypoint capture error distance. An example of this can be seen by comparing waypoints 1 and 6. Waypoint 1 requires the vehicle to turn nearly a full 360 degrees, thus having a massive waypoint capture error distance. Waypoint 6 requires the vehicle to turn 100 degrees, thus having a smaller waypoint capture error distance. This maximum waypoint capture error distance recorded was 3.38m, which occurs at waypoint 1.

2.7). Testing Cross-Track Error

The cross-track error is considered to be the perpendicular distance that a robot/vehicle has varied from the desired path. The lower the cross-track error, the faster the mission completion time. The Root Mean Square (RMS) error is used to find the average distance that the vehicle is from the desired path. The spike in the Cross-Track Error should be when a waypoint capture error occurs, as the vehicle would take time to get back on the desired route. An example of the Cross-Track Error can be seen in Figure 2.8 below.

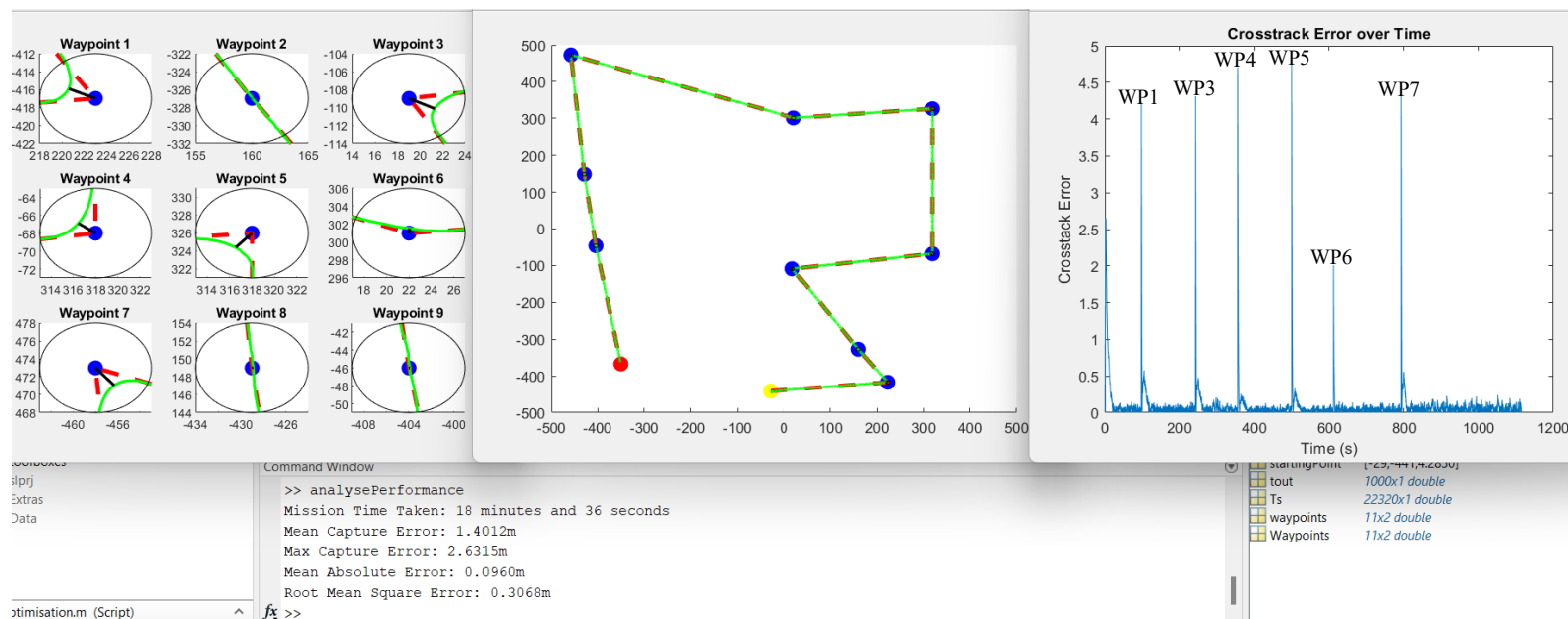


Figure 2.8: Testing effects waypoint capture error has on cross-track error

As shown above, the spikes in the cross-track error graph correspond to the waypoint capture error, especially when the vehicle has to turn more than 90 degrees. Waypoints 2, 8, and 9 do not show up on the cross-track error graph as the deviation from the desired path at those waypoints is minimal to none. Also seen above is the RMS error, which is 0.3068m. This error is higher than expected, as a normal cross-track error would be around 0.1m. The spikes in this graph affect the RMS cross-track error distance as they are significantly more than the normal distance.

3). Waypoint Optimisation

3.1). Brute Force Approach

A Brute Force (BF) algorithm is exactly what it says. The algorithm systematically searches all the possible solutions for a problem and selects the best result [5]. The best result depends on what section of a system needs to be optimised, such as minimum distance or speed. In this BF approach the minimum distance was optimised.

3.2). Branch and Bound Approach

A Branch and Bound (BNB) algorithm is an algorithm that is very similar to the BF algorithm but finds the best solution using a different technique. The BNB algorithm still creates every possible solution, but then changes the way it finds the best solution. Instead of analysing and completing every possible solution, the algorithm will get rid of the 'branches' that won't be the best solution. [6]

3.3). Nearest Neighbour Approach

The nearest neighbour (NN) Algorithm is also another self-explanatory algorithm as it is stated in the name of the algorithm. Unlike the other algorithms discussed previously, this algorithm does not create all possible solutions. This algorithm calculates the distance between the start position and the other waypoints. Once it finds the minimum distance between the start point and a waypoint, the vehicle moves to this position. This process of finding the minimum distance and moving keeps happening until all waypoints have been met. [7] The code for the NN algorithm applied in this assignment can be seen in Figure 3.3 below.

```
function [Waypoints, totalDistance] = nearestNeighbourV1(waypoints)
    Waypoints = [waypoints(1,:); zeros(size(waypoints,1)-1,2)];
    waypoints(1,:) = [];
    totalDistance = 0;
    for i = 2:size(Waypoints,1)
        bestDistance = inf;
        for j = 1:size(waypoints,1)
            Distance = sqrt((waypoints(j, 1) - Waypoints(i-1, 1))^2 +
(waypoints(j, 2) - Waypoints(i-1, 2))^2);
            if Distance < bestDistance
                bestDistance = Distance;
                Idx = j;
            end
        end
        totalDistance = totalDistance + bestDistance;
        Waypoints(i,:) = waypoints(Idx,:);
        waypoints(Idx,:) = [];
    end
end
```

Figure 3.3: Code for Nearest Neighbour Algorithm (MATLAB)

This code starts by iterating through the different coordinates. By the end of the first iteration, it would find the closest waypoint, take it out of the waypoints remaining, and change the 'Idx' or index to the new waypoint the algorithm is searching from. It should also add the distance to the total distance. There is one input called 'waypoint' that should be a two by ten or eleven double array that represents the waypoint coordinates. The outputs of this function are waypoints and the total distance undertaken by the vehicle.

3.4). Waypoint Approaches Comparison

Multiple tests were completed to test the different ways the vehicle could solve the problem. One of these tests can be seen below in below in Table 3.1. (See Appendix B-E for pictures of the path taken by each algorithm and command window). The 'unoptimised' algorithm just follows the waypoints in the order they are given, which is unknown as the process would not require any decision making, this unknown.

Algorithm	Distance (meter)	Computational Time (second)
Unoptimised	4422.221	Unknown
Brute Force	1972.2936	1.6861
Branch and bound	1972.2936	0.0246
Nearest Neighbour	1999.1430	0.0008

Table 3.1: Table comparing algorithms based on Test 1

As seen in the table above, the BF and BNB algorithms seem to get the same distance but different computation times. This is because the BNB deletes the other undesirable possible solutions. The NN algorithm shows that it has the fastest computational time but is not the best for minimising the total distance. Although the different algorithms are trying to optimise certain parameters, there is an instance where they get the same distance but different computation times. (See Appendix F-H for test 2 and the results)

4). References

- [1] T. S. Bruggemann, J. J. Ford and R. A. Walker, “Control of Aircraft for Inspection of Linear Infrastructure,” QUT, June 2010. [Online]. Available: <https://eprints.qut.edu.au/40010/1/c40010a.pdf>. [Accessed 05 September 2024].
- [2] J. Ford and D. Quevedo, “EGH446 Autonomous Systems: Week 5 Lecture - Introduction to Kalman Filters,” QUT, 19 August 2024. [Online]. Available: https://canvas.qut.edu.au/courses/17127/pages/5-dot-2-lecture-3?module_item_id=1684761. [Accessed 05 September 2024].
- [3] L. Mejias, “EGH446 Autonomous Systems - Week 4 Lecture: Motion Control,” QUT, 12 August 2024. [Online]. Available: https://canvas.qut.edu.au/courses/17127/pages/4-dot-2-lecture-3?module_item_id=1740683. [Accessed 06 September 2024].
- [4] freeCodeCamp, “Brute Force Algorithms Explained,” freeCodeCamp, 06 January 2020. [Online]. Available: <https://www.freecodecamp.org/news/brute-force-algorithms-explained/>. [Accessed 06 September 2024].
- [5] geeksforgeeks, “Branch and Bound Algorithm,” geeksforgeeks, 22 February 2024. [Online]. Available: <https://www.geeksforgeeks.org/branch-and-bound-algorithm/>. [Accessed 06 September 2024].
- [6] IBM, “What is the KNN algorithm?,” IBM, [Online]. Available: <https://www.ibm.com/topics/knn>. [Accessed 06 September 2024].

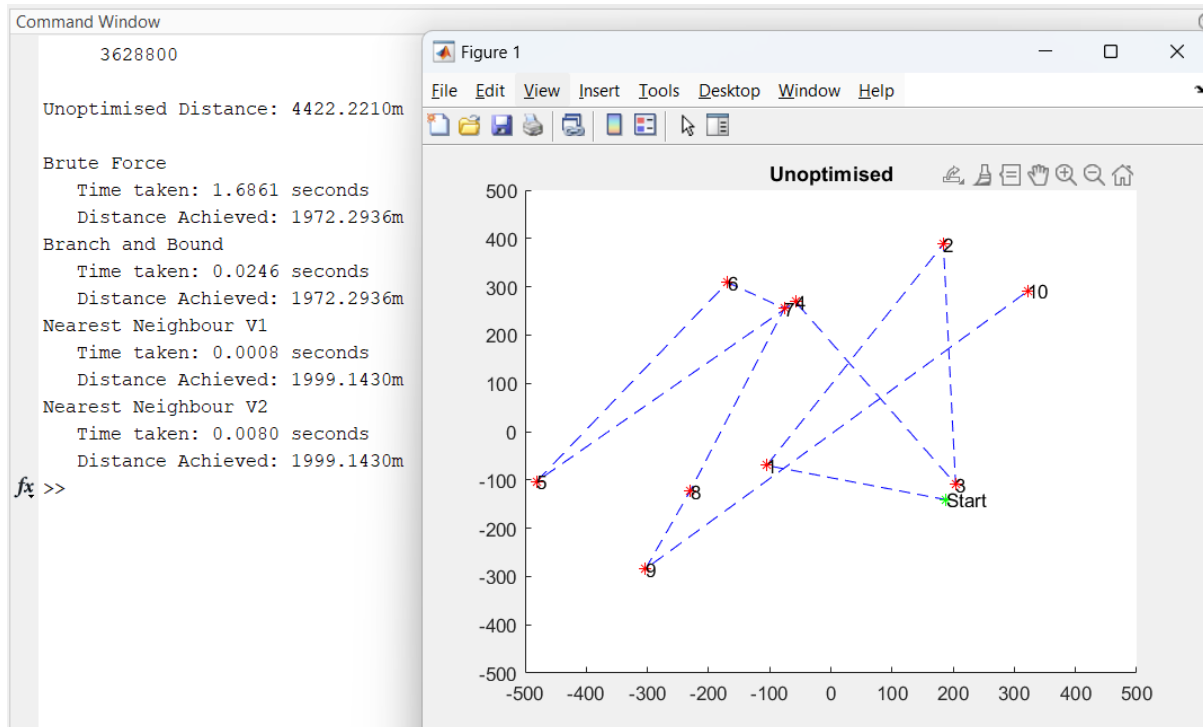
5). Appendices

Appendix A: 'branchAndBound' function code

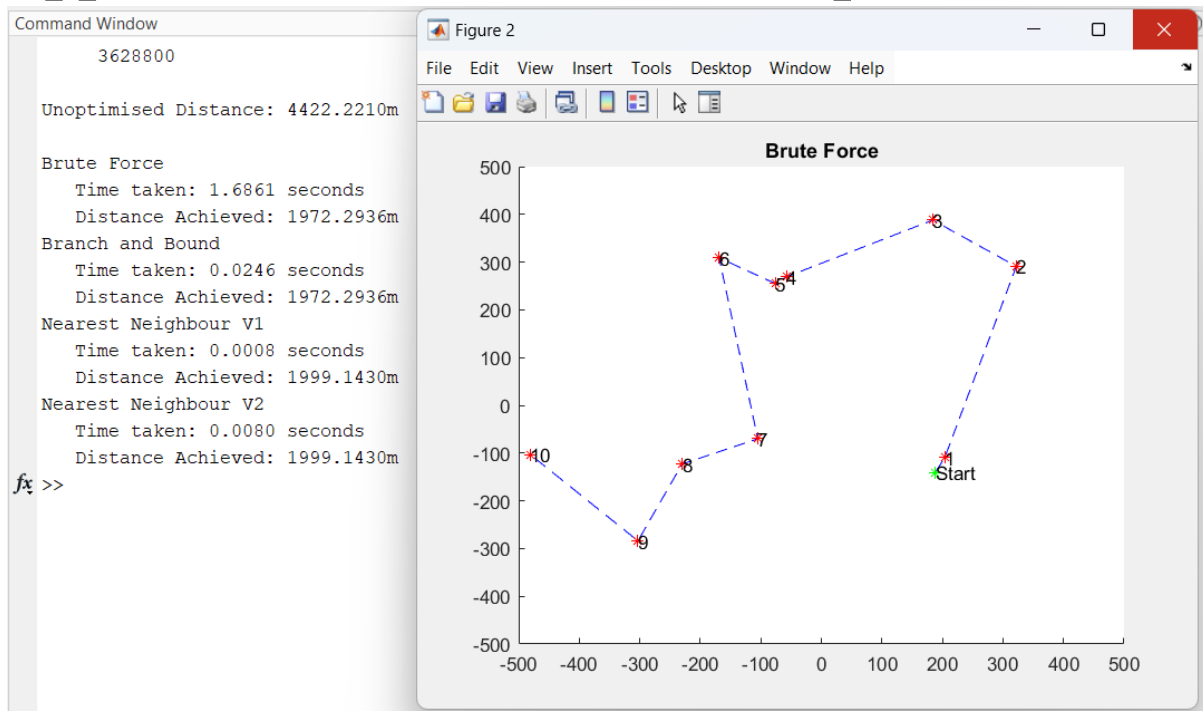
```
function [bestPath, bestDistance] = branchAndBound(currentPath, unvisited,
distanceMatrix, currentDistance, bestDistance, bestPath)
    if isempty(unvisited)
        if currentDistance < bestDistance
            bestDistance = currentDistance;
            bestPath = currentPath;
        end
        return;
    end
    for i = 1:length(unvisited)
        newPath = [currentPath, unvisited(i)];
        newDistance = currentDistance + distanceMatrix(currentPath(end),
unvisited(i));
        remaining = unvisited;
        remaining(i) = [];
        lb = calculateLowestBound(newPath, remaining, distanceMatrix);
        if lb < bestDistance
            [bestPath, bestDistance] = branchAndBound(newPath, remaining,
distanceMatrix, newDistance, bestDistance, bestPath);
        end
    end
end

function lb = calculateLowestBound(visited, unvisited, distanceMatrix)
    lb = 0;
    for i = 2:length(visited)
        lb = lb + distanceMatrix(visited(i),visited(i-1));
    end
    if length(unvisited) > 1
        edge = distanceMatrix(visited(end),unvisited);
        lb = lb + min(edge(edge>0));
        edges = distanceMatrix(unvisited, unvisited);
        edges = edges(edges>0);
        lb = lb + sum(mink(edges, ((length(unvisited)-1)*2)))/2;
    end
end
```

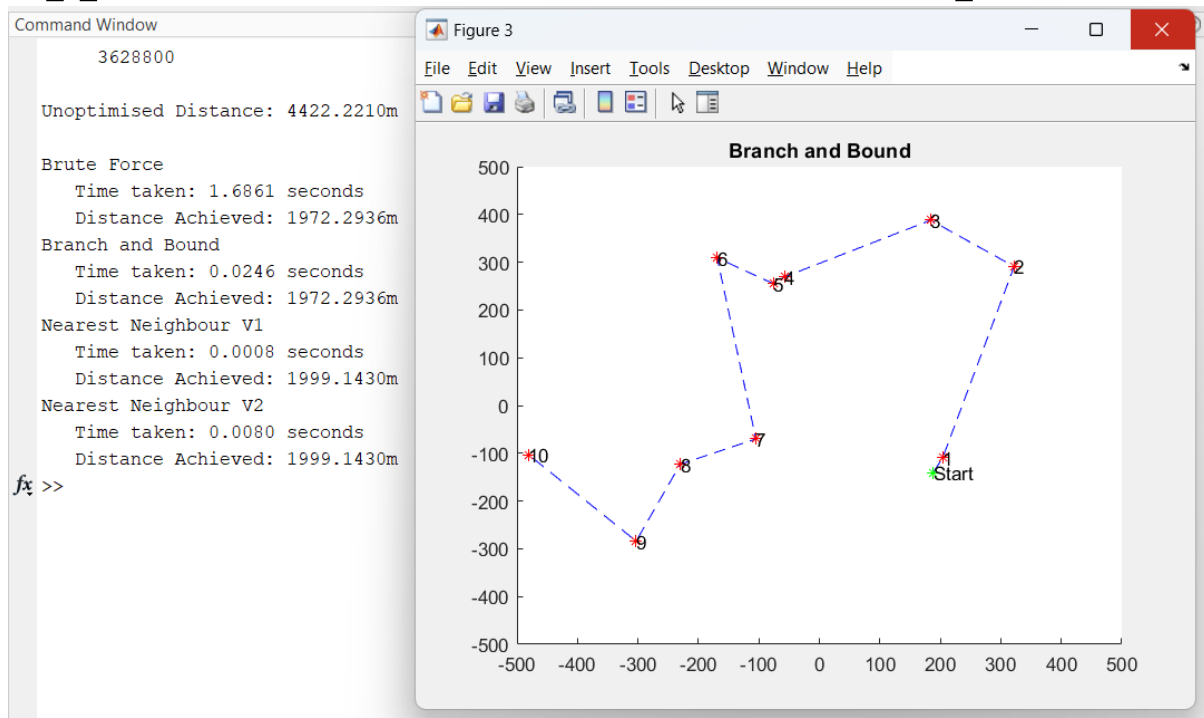
Appendix B: Test 1 Unoptimised path



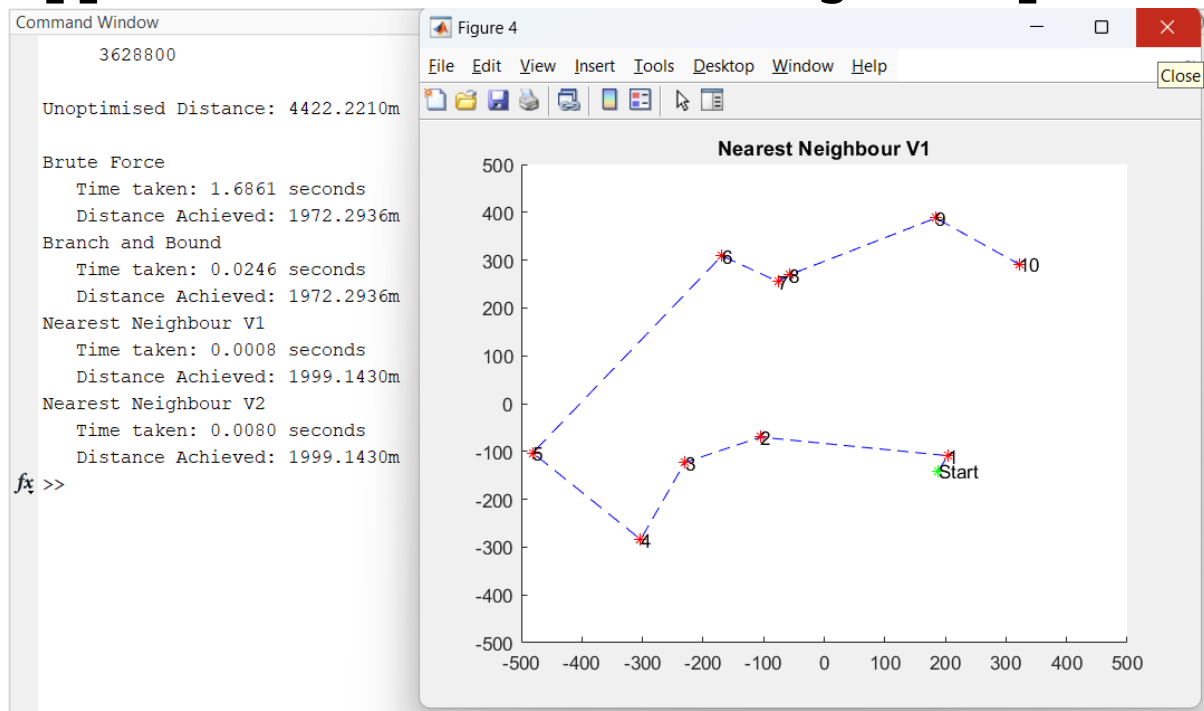
Appendix C: Test 1 Brute Force path



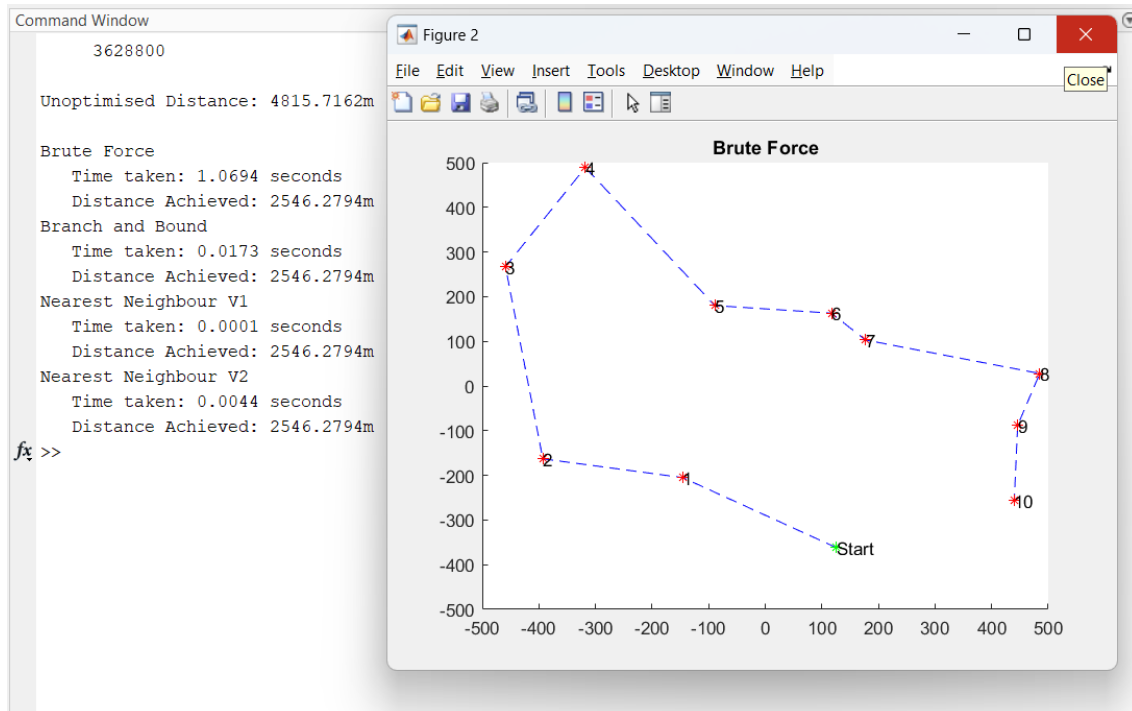
Appendix D: Test 1 Branch & Bound path



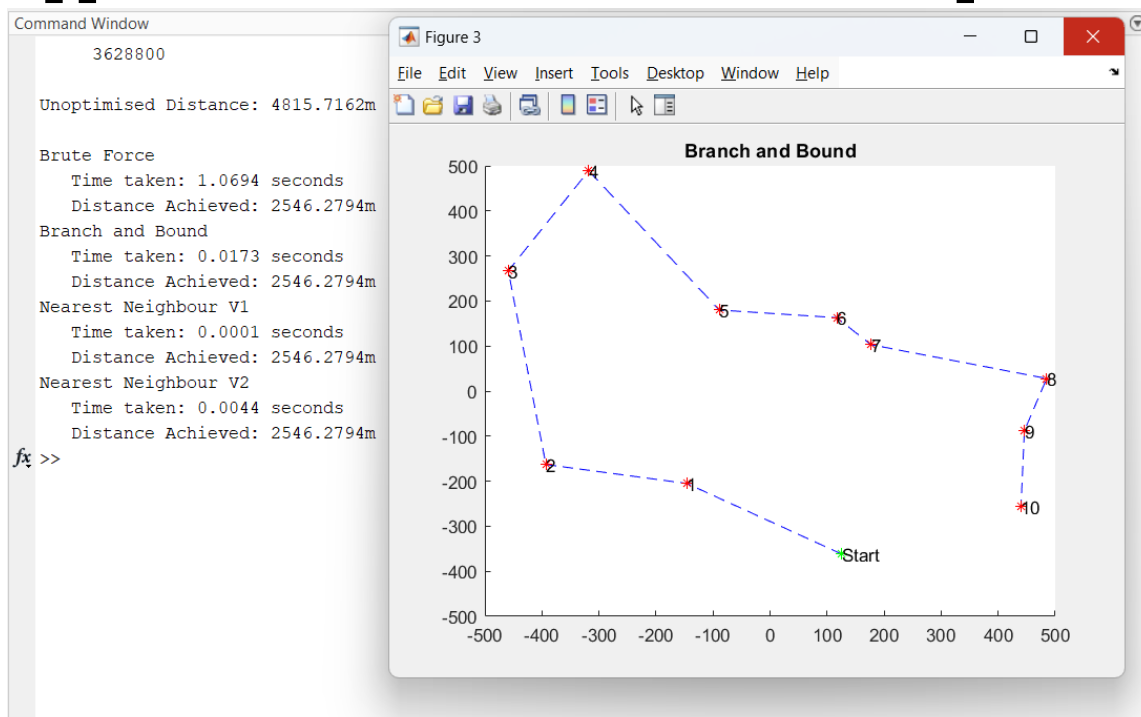
Appendix E: Test 1 Nearest Neighbour path



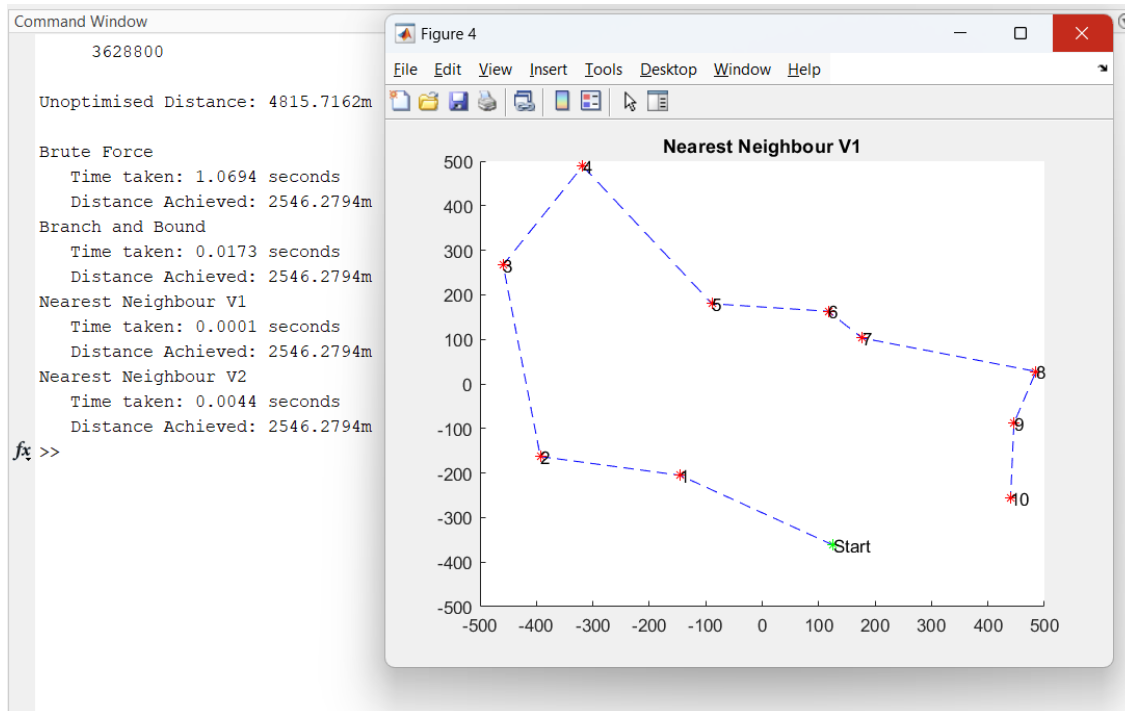
Appendix F: Test 2 Brute Force path



Appendix G: Test 2 Branch & Bound path



Appendix H: Test 2 Nearest Neighbour path



Appendix I: 'transform' function code

```
function vect = transform(Pose, Target)
    SE2 = [cos(Pose(3)), -sin(Pose(3)), Pose(1); sin(Pose(3)), cos(Pose(3)),
    Pose(2); 0, 0, 1];
    vect = inv(SE2)*[Target(1); Target(2); 1];
    vect = vect([1,2],:);
end
```